北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

第4章　类和对象

前言：面向对象vs面向过程

面向过程：

首先分析问题的步骤，按步骤实现，以编程语言为本

面向对象：

将客观事物看作具有状态和行为的对象，通过抽象找出同一类对象的共同状态和行为，构成类。可以提高程序的复用性和可扩展性。

面向对象

- 面向对象的程序设计将数据和对数据的操作行为封装在一起，作为一个相互依存、不可分割的整体--类。
- 类是抽象的数据类型，用类创建对象。
- 面向对象的显著特点包括：封装性、继承性和多态性。

类和对象

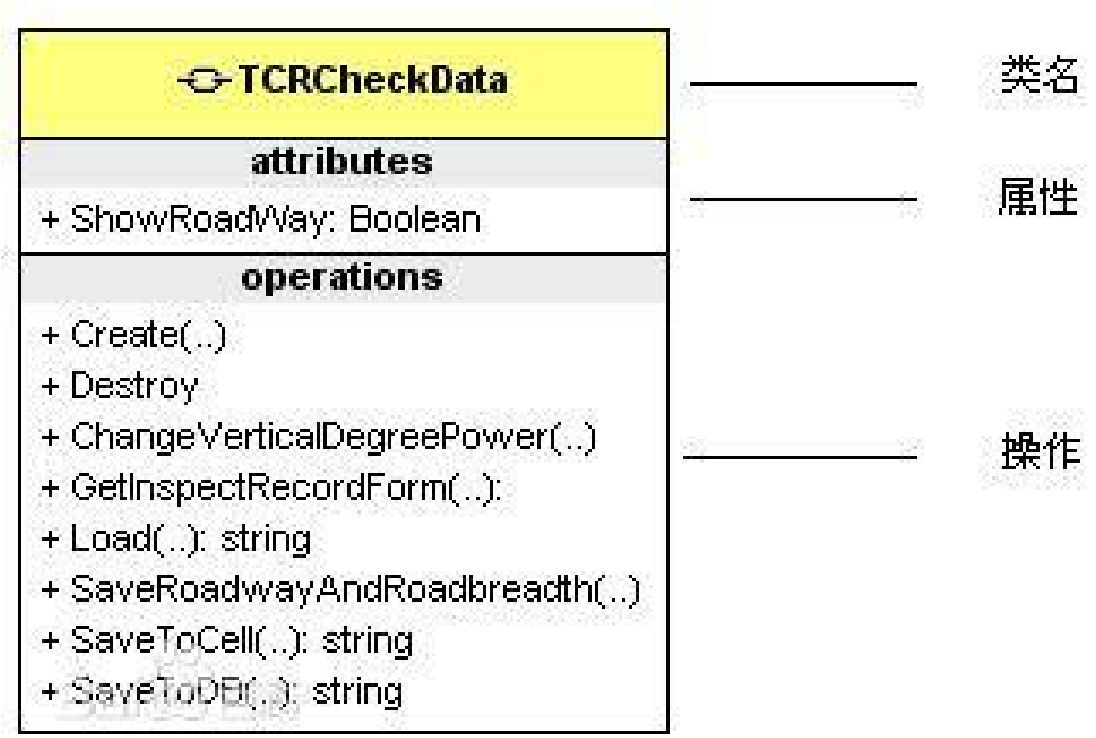
- 类（class）：OOP中**抽取同类实体的共同性自定义一种数据类型称之为类。**

- 对象就是一个类的具体的实例。

类与对象

- 类是对某一类事物的描述，是抽象的、概念上的定义；对象是实际存在的该类事物的每个个体，因而也称实例（instance）

UML中类的表示


TCRCheckData
attributes
+ ShowRoadWay: Boolean
operations
+ Create(..)
+ Destroy
+ ChangeVerticalDegreePower(..)
+ GetInspectRecordForm(..):
+ Load(..): string
+ SaveRoadwayAndRoadbreadth(..)
+ SaveToCell(..): string
+ SaveToDB(..): string
类名
属性
操作

本章知识点

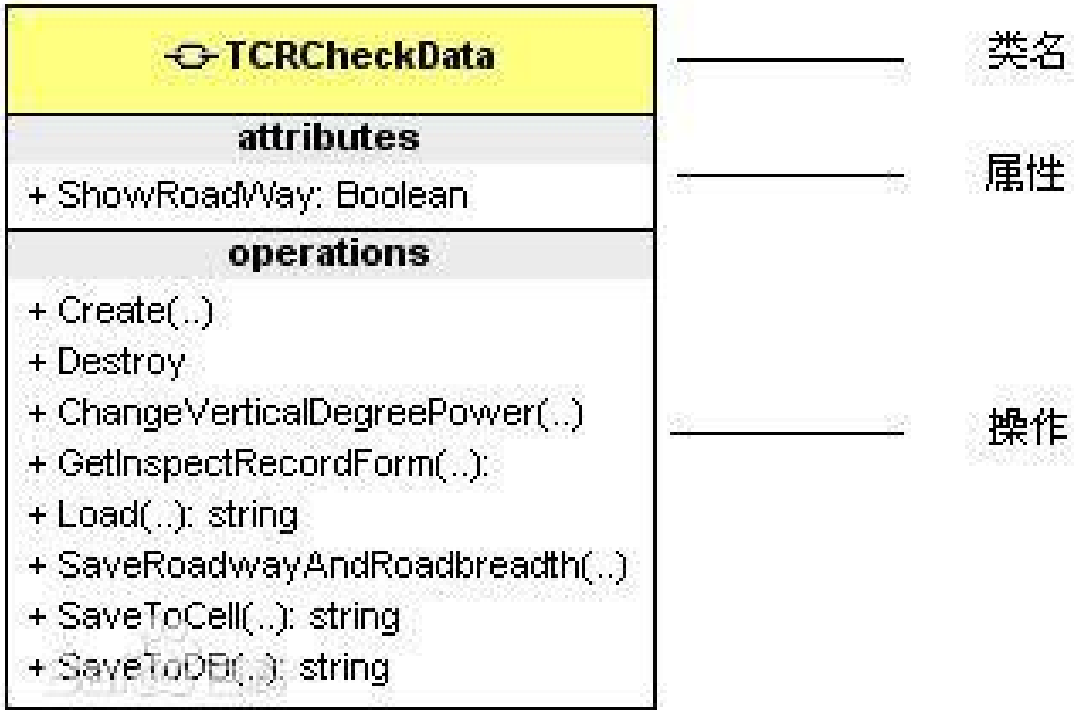
- 1Java概述
- 2类的创建
- 3创建和使用对象
- 4构造器初始化对象
- 5变量和常量
- 6包的创建和使用
- 7基本类型与引用类型的转换
- 8操作符与对象
- 9对象初始化小结1

一种比较”纯粹“的面向对象的编程语言

Everything in Java is an object, even a Java program.

4.2 类的创建

类作为一个抽象的数据类型，用来描述相同类型的对象。



4.2.1 类的设计

- 酒店类的设计，酒店有若干客房，向客户提供查询、入住、退房等功能。

名词 → 对象

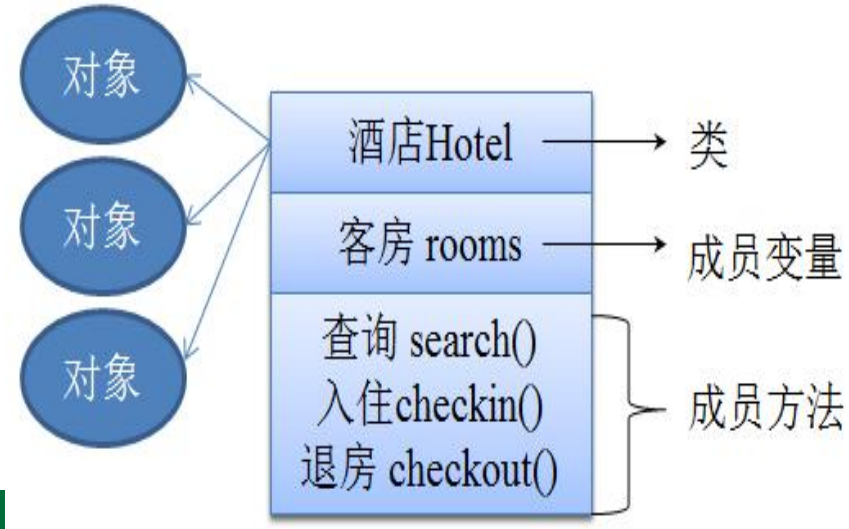
动词 → 对象的行为

酒店 客房

查询

入住

退房



4.2.2 使用class定义类

定义类的语法格式

```
[类的修饰符] class 类名 [extends 父类名]
{
    ……        //类体
}
```

类体包括数据成员（属性）和方法

4.2.2 使用class定义类

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 1. 数据成员（）

[修饰符] 数据类型 成员名[=默认值];

例：

- **private** String hotelName;
- **private** String[][] rooms
- private int numRoom;

数据类型	关键字	缺省数值
布尔型	boolean	false
字符型	char	'\u0000'
字节型	byte	0
短整型	short	0
整型	int	0
长整型	long	0
浮点型	float	0.0F
双精度型	double	0.0D
引用类型	类、接口	null

4.2.2 使用class定义类

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

2. 方法

- 一般是对类中的数据成员进行操作
- set/get方法
  - 存：设置private成员变量的取值—setXxxx(xxx){}
  - 取：读取private成员变量的取值—xxx getXxxx(){}
- 业务方法

4.2.2 使用class定义类

方法主要包括：

方法名　参数列表　返回值类型　方法体

```
返回值类型 方法名( /* 参数列表 */ ) {
        /* 方法体 */
    }
```

形参个数可变的方法：

类型...形参名

本质是数组，等价于 类型[] 形参名

好处是更方便，即可传入多个参数，也可以传入数组

注意：

类型...形参名，只能放在参数列表的最后；

一个方法只能有一个数可变的形参。

4.2.2 使用class定义类

【例4-1】定义一个酒店类Hotel。

（1）类的访问控制符
（2）数据成员（成员变量）：记录对象性质和状态的变量
（3）数据成员的set和get方法
（4）构造方法

example.ex1.hotel

使用关键字new创建对象

标识符：对象的应用，该标识符对应的变量的类型为引用类型。

```
public static void main(String[] args){
            Hotel hotel1 = new Hotel("MiniHilton", new String[10][20]);
            Hotel hotel2 = new Hotel("MiniStarwood", new String[15][20]);

            int res = hotel1.compareTo(hotel2);

        }
```

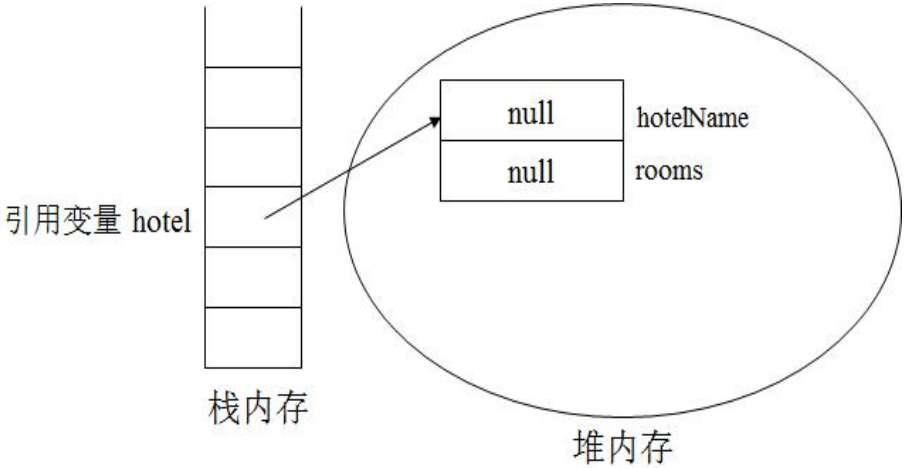
4.3.1 对象和引用的关系

- 对象：通过new关键字调用某个构造方法创建，为该对象分配内存空间，并按照构造方法的方法体对对象的数据成员赋初值，创建好的对象在堆内存中。

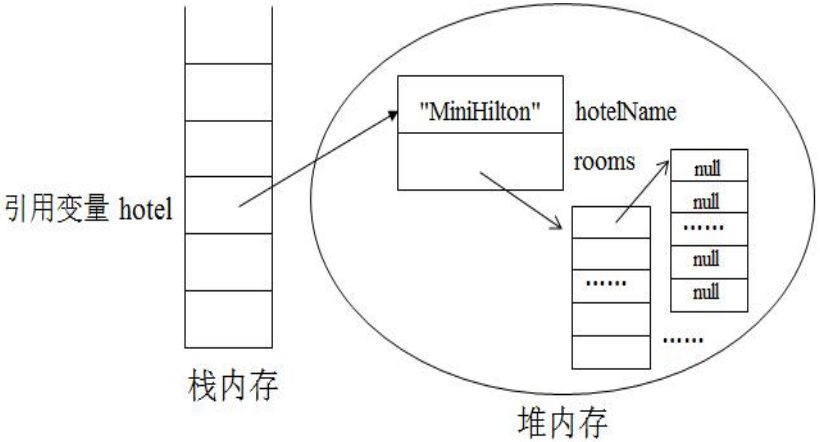
- 引用：Java不允许直接访问堆内存中的对象，只能通过标识符操作该对象，标识符存放在栈内存中

4.3.1 对象和引用的关系

- 【例】酒店问题中的对象和引用。



Hotel hotel = new Hotel();



Hotel hotel = new Hotel("MiniHilton", new String[10][20]);

4.3.1 对象和引用的关系

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- Java中的数据存放：

  寄存器。

  栈

  堆

  常量值存放在程序代码内部

  非RAM存储

4.3.2 对象的使用

- 如果想使用对象，必须先创建对象，再使用该对象。

使用属性： 对象名.属性名

使用方法： 对象名.方法名（实参列表）

4.3.3 static关键字

如果类中的数据成员和方法用关键字static声明，则不需必创建对象直接使用类名即可调用。

```
class StaticTest{
        static int i = 47;
    }
StaticTest st1 = new StaticTest( );
```

静态变量访问方式：类名.数据成员；

st1.i　　**StaticTest.i**

4.3.3 static关键字

- static数据成员：为类的对象所共享的数据
- static方法：不必创建对象直接使用类名即可调用。

```
Math.PI
Math.random()
Math.sin()
```

static修饰的数据成员
不建议使用对象名.成员变量

4.3.3 static关键字

易考

- 在static方法中不允许使用非static成员和方法
- 在非static方法中既可以使用非static成员，也可以使用static成员

生命周期

StaticErr.java

【例4-3】定义含有static数据成员的Person类。

4.3.4 static代码块

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 如果在加载类时希望先进行一些特殊的初始化动作，可以使用static定义一个代码块，将期望最早执行的初始化任务写在代码块中。
- StaticInit.java
- 【例4-5】静态代码块示例。

静态代码块一般用来在类加载以后初始化一些静态资源时候使用，如：加载配置文件。类加载：第一次主动使用该类时（定义变量不算）

4.3.5对象的作用域

由new创建的对象一直存在直至被垃圾回收器销毁

```
{
    String s=new String("world")
}
```

但标识符s的作用域为{}内，{}外s不可用

4.4 构造器

Java通过构造器constructore保证对象的初始化

构造器名与类名一致

构造器没有返回值

构造器可以重载

Sample : Overloading.java

DefaultConstructor.java

如果没有定义构造器，java编译器才会自动为类提供默认构造器。

4.4.1 this引用

- this：代表一个引用，指向正在调用该方法的当前对象。

```
hotel.setHotelName("MiniStarwood");
```

```
public void setHotelName(String hotelName) {
        this.hotelName = hotelName;
}
```

Sample : Leaf.java

```
public class Leaf {
  int i = 0;
  Leaf increment() {
    i++;
    return this;
  }
  void print() {
    System.out.println("i = " + i);
    String s= new String ("print");
  }
  public static void main(String[] args) {
    Leaf x = new Leaf();
    x.increment().increment().increment().print();
  }
}
```

类成员变量

局部变量

4.4.2 构造器调用

In a constructor, the **this** keyword takes on a different meaning when you give it an argument list ---- it calls the constructor that matches that argument list.

this（参数列表）

必须放在构造器的第一行

Sample : Flower.java

练习

【题目】编写一个学生类，包括学号、姓名、性别、年龄和记录学生总数的数据成员。

（1）编写合理的重载构造方法。

（2）编写各数据成员的set、get方法。

（3）编写测试类创建几个学生，打印他们的信息。

（4）在测试类中创建学生数组存储学生对象，打印数组中每个学生的信息。

4.5 变量和常量

- 数据成员/局部变量？
- 数据成员：属于每个对象/属于类？
- 方法局部变量/代码块局部变量？

- 【例4-4】变量的使用规则示例。

4.5.1 变量的使用规则

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 代码段1：

```
public class TestScope1 {
    static int i;          // 定义一个类数据成员作为循环变量
    public static void main(String[] args) {
        for(i=0; i<10; i++){
            System.out.println(i);
        }
    }
}
```

- 代码段2：

```
public class TestScope2 {
    public static void main(String[] args) {
        int i;    // 定义一个方法局部变量做循环变量
        for(i=0; i<10; i++){
            System.out.println(i);
        }
    }
}
```

- 代码段3：

```
public class TestScope3 {
    public static void main(String[] args) {
        for(int i=0; i<10; i++){    // 定义一个代码块局部变量作为循环变量
            System.out.println(i);
        }
    }
}
```

4.5.2 类常量的定义

- final：放在变量声明前，表示该变量一旦被赋值后，就不能再改变其取值，即通常意义上的符号常量。
- static：如果这个常量属于类的每一个对象，则可以在其定义前加上修饰。

- static final ……

4.5.2 类常量的定义


```
public class Hotel {
    private static final int HEIGHT=10;
//层数
    private static final int WIDTH=12;
//客房数
            private String hotelName;  //
酒店名
    private String[][] rooms;    //酒店客
房
    public Hotel(){
        rooms = new
String[HEIGHT][WIDTH];
```

4.7 java 文件组织--包

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- “包” 方式：包与磁盘的文件系统结构相对应，一个包就相当于一个文件夹，包中的类相当于文件夹下的文件。

- Java用为类定义不同的包，即定义不同的存储位置的方式解决同名类的冲突。同时包也提供了类的分类管理，使类可以按功能、来源等分为不同的集合，便于组织和使用。

4.6 包

- java.lang：这个包下包含了Java语言的核心类，如String、Math、System等，这个包下的类在程序运行时自动导入。
- java.util：包含了大量实用的工具类和集合等，如Scanner、Date、Arrays、List等。
- java.text：包含了一些与Java格式化相关的类

4.6.1 包的创建

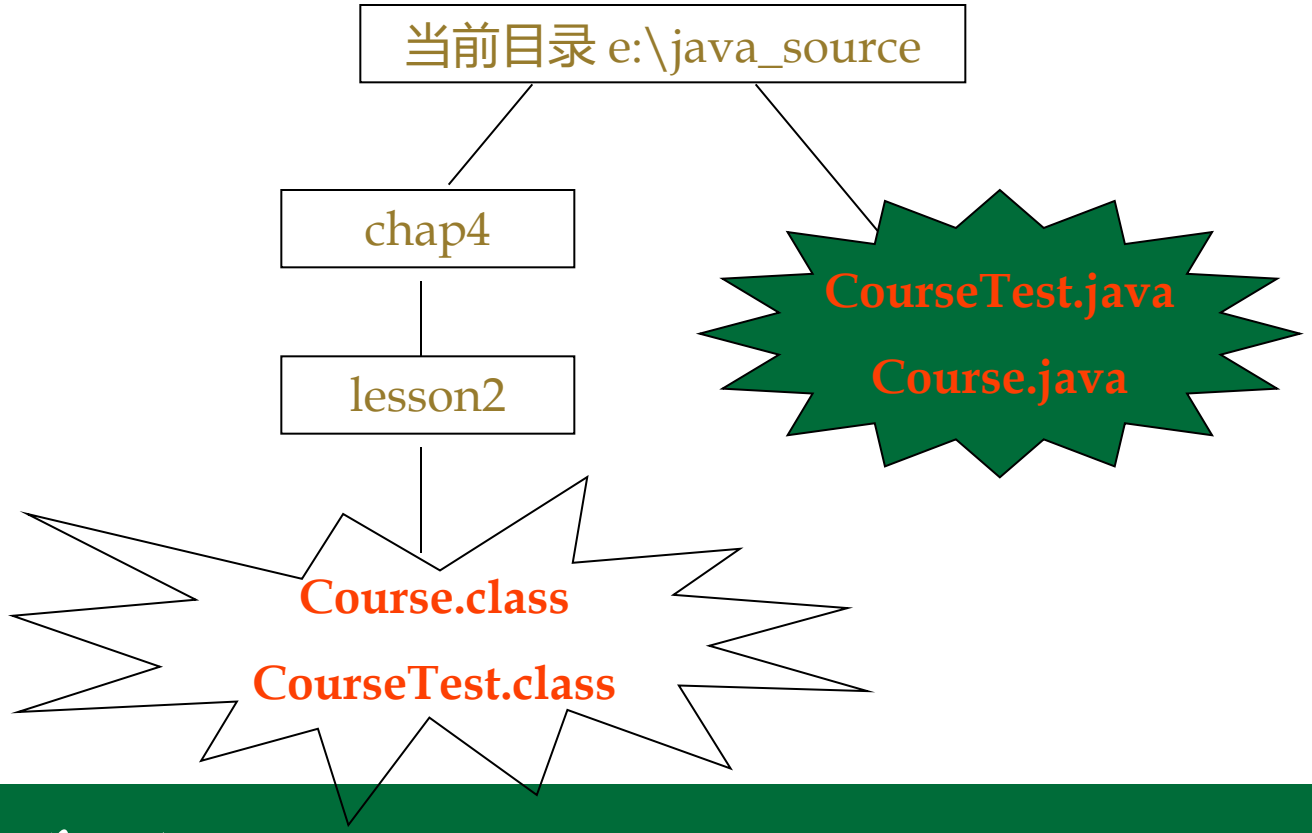
- 创建包：Java文件的第一条语句
- 语句格式

  package 包名[.包名[.包名]…];
- Java包的名字都是由小写字母组成，“.”指明包（文件夹）的层次。

例如： pakage chap4;
package chap4.lesson1;
包 子包

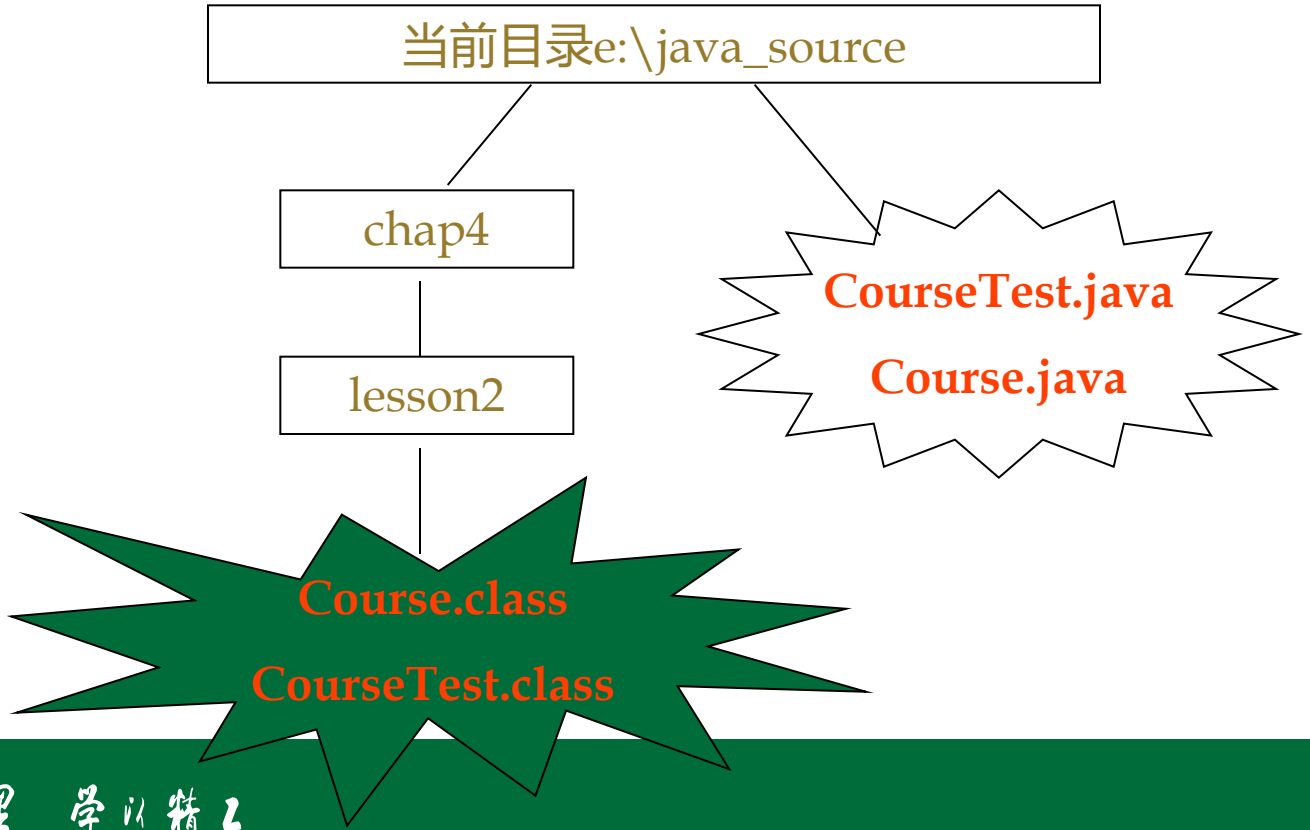
命令行方式下的编译和执行

- javac -d . Course.java
- javac -d . CourseTest.java

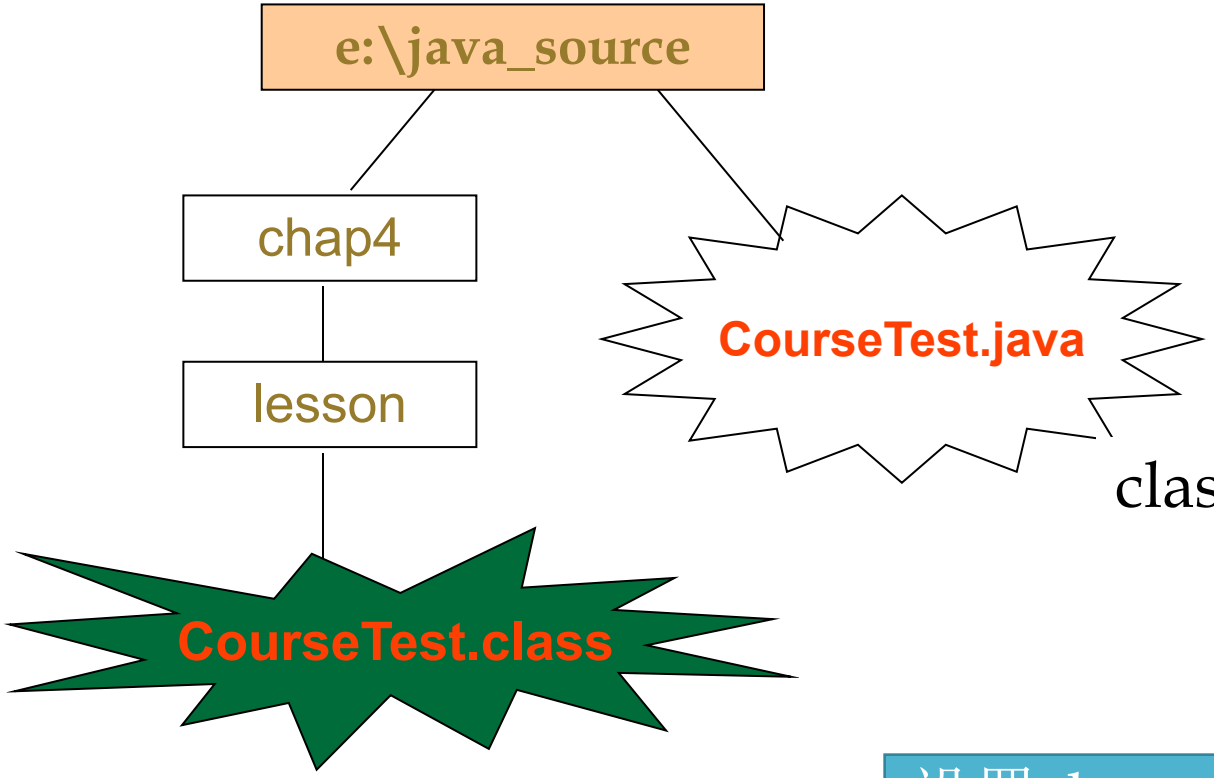


命令行方式下的编译和执行

- 带包定义的类的执行
  - 需要在包的上级目录执行类的class文件
  - java　chap4.lesson2.CourseTest



命令行方式下的编译和执行



- 设置环境变量classpath：指定寻找类字节码的路径—包的上级文件夹。

classpath=e:\java_source;

设置classpath后，在任意路径都可以执行包下面的类。

4.6.2 类的导入

- 同一个包中的类可以互相访问：无需导入
- 一个类要访问位于另一个包中的类（设这个类允许被访问）：**通过import语句导入类**

- import不能自动引入包的子包，必须用显式声明的方式导入子包。

```
import java.awt.*;
import java.awt.event.*;
```

4.6.2 类的导入

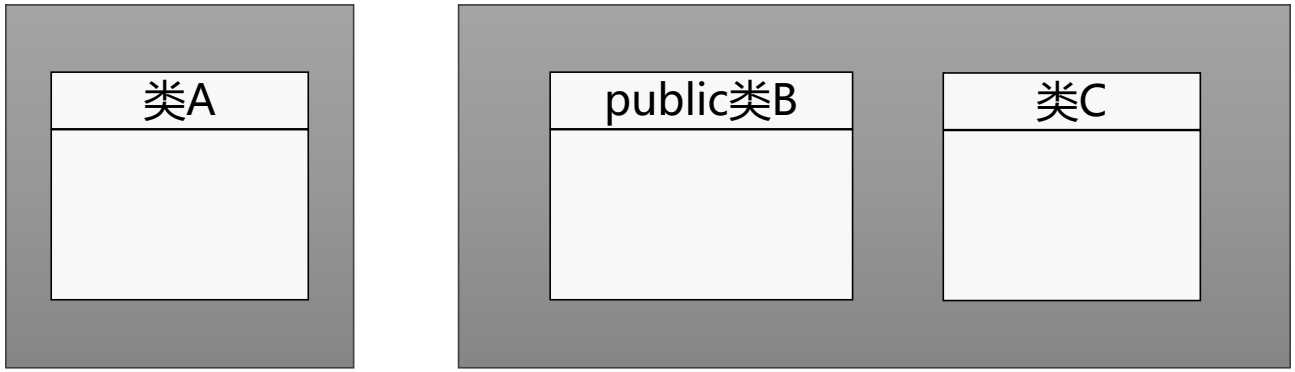
北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 类的访问控制：public | 缺省

[类修饰符] class 类名

- public：该类**可以**被所有类访问（包内或包外均可）
  - 如果使用者与被使用者**在同一个包**下：直接使用
  - 如果使用者与被使用者**不在同一个包**下：通常先导入，再使用（注意，这个类必须是 public 的）

4.6.2 类的导入



类A可以访问类B吗? ✓

类A可以访问类C吗? ✗

类B可以访问类A吗? ✗

类B可以访问类C吗? ✓

类C可以访问类A吗? ✗

类C可以访问类B吗? ✓

通过import 实现条件编译

conditional compilation allows you to change a switch and get different behavior without changing any other code.

Sample :

Assert.java TestAssert.java

注意默认包：工程下src包下的类默认在同一个包中。Sample : ToolTest.java

1. 编译

```
d:\java_source> javac -d . Hotel.java
```

参数1“-d”：指定编译时在文件系统中创建与包对应的文件夹结构，必须使用。

参数2：指定类文件（包结构+class文件）的存储位置。该参数由程序设计者按需指定。

参数3：指定被编译的源文件。一定要保证按照路径可以找到被编译的源文件。

4.6.3 含包定义的类的编译及执行

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

2. 执行

- 完整的类名：包名.类名

- 执行class文件
  - 在当前路径下可以按照包名结构找到被执行的类文件
  - 必须使用完整的类名

4.6.4 包中的java文件

一个包中可以有多个java文件

一个java文件中可以写多个类，但必须且只有一个类与java文件同名。

编译一个java文件，文件中的每个类都会被编译为一个class 文件。

4. 6程序示例

- Sample : Vector.java　List.java

- Sample : LibTest.java

- 创建自己的类库

Sample :　com.bruceeckel.tools.P.java
ToolTest.java

4. 7包装器类

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 基本类型有相对应的包装器类 **“wrapper” classes**.

```
char c = 'x';

Character C = new Character('x' );

Character C = new Character( c );
```

- **此外java还提供满足高精度运算的: BigInteger** and **BigDecimal类**

4.7包装器类

Primitive type	Size	Minimum	Maximum	Wrapper type
boolean	—	—	—	Boolean
char	16-bit	Unicode 0	Unicode $2^{16}-1$	Character
byte	8-bit	-128	+127	Byte
short	16-bit	-2^{15}	$+2^{15}-1$	Short
int	32-bit	-2^{31}	$+2^{31}-1$	Integer
long	64-bit	-2^{63}	$+2^{63}-1$	Long
float	32-bit	IEEE754	IEEE754	Float
double	64-bit	IEEE754	IEEE754	Double
void	—	—	—	Void

4.8操作符与对象

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 赋值运算符

- 关系运算符

4. 8. 1赋值运算符与对象

赋值运算可以对引用类型的数据操作

When you assign “from one object to another” you’re actually copying a reference（引用） from one place to another.

This means that if you say **C = D** for objects, you end up with both **C** and **D** pointing to the object that, originally, only **D** pointed to. ---- Aliasing（别名）

- 基本类型 “=”
- when you assign primitives, you copy the contents from one place to another .

a = b

- the contents of **b** are copied into **a**. If you then go on to modify **a**, **b** is naturally unaffected .

方法调用中的别名现象

Sample : PassObject.java

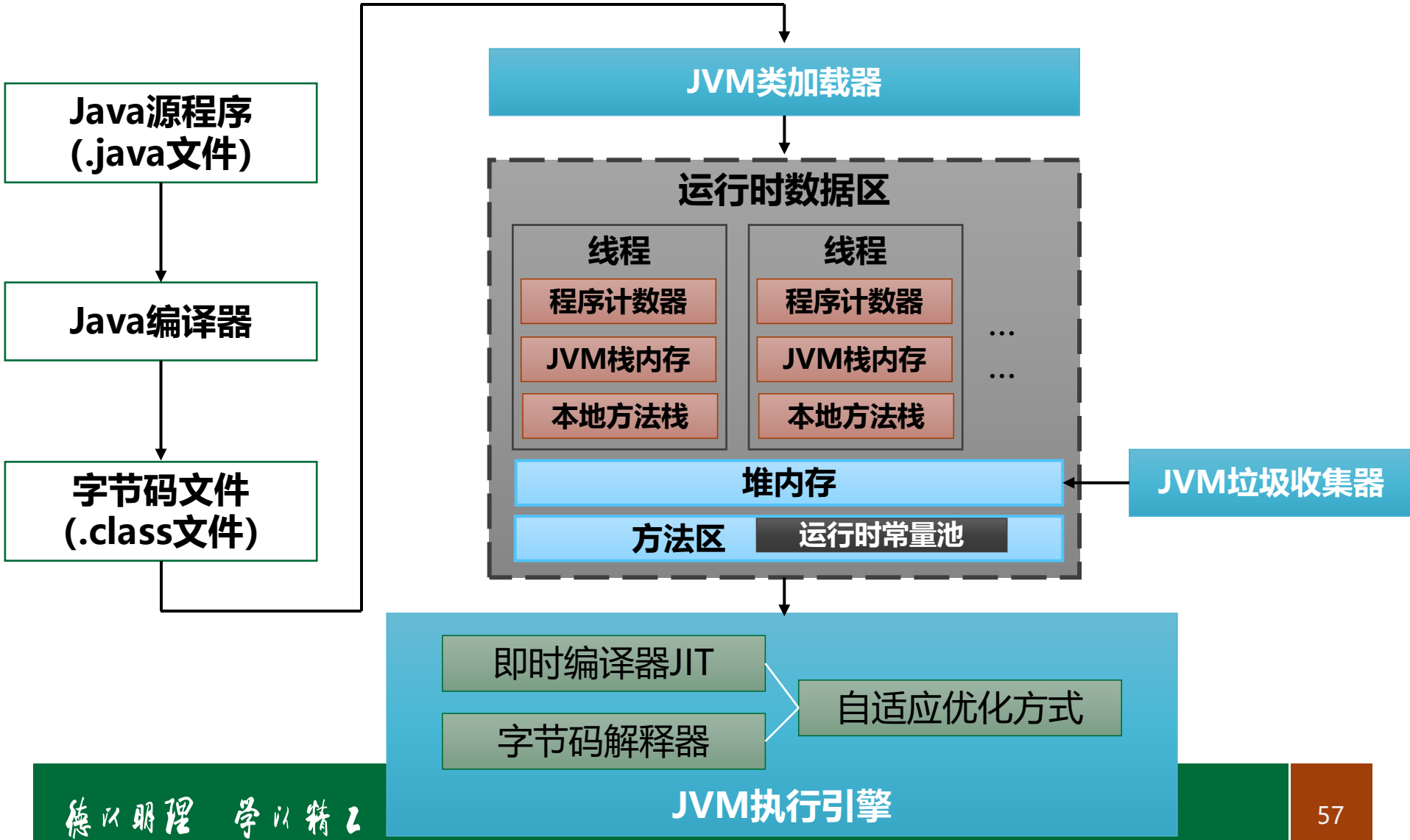
- assign to objects:

copying a reference from one place to another

This phenomenon is often called *aliasing（别名现象）*

Sample : Assignment.java

JVM栈内存(stack)和堆内存(heap)



4.8.1赋值运算符与对象

程序中的每个方法被执行时都会创建一个栈帧用来存储该方法的局部变量等信息，包括各种基本类型的变量和数组/对象的引用变量。

```
public class Ex215 {
    public static void main(String[] args) {
        printPyramid('g');
    }
    public static void printPyramid(char c){
        for(char row='a'; row<=c; row++){
            for(int i=1; i<=c-row; i++){
                System.out.print(" ");
            }
            for(char ch='a'; ch<=row; ch++){
                System.out.print(ch);
            }
            for(char ch=(char)(row-1); ch>='a'; ch--){
                System.out.print(ch);
            }
            System.out.println();
        }
    }
}
```

4.8.2关系运算符与对象

==比较对象的引用是否相同

Sample :

Equivalence.java EqualsMethod.java
EqualsMethod2.java

object类中的 **equals( )** 比较对象的引用是否相同，但Java中提供的类中大多数对**equals( )**进行了重写 *override，实现值的比较*

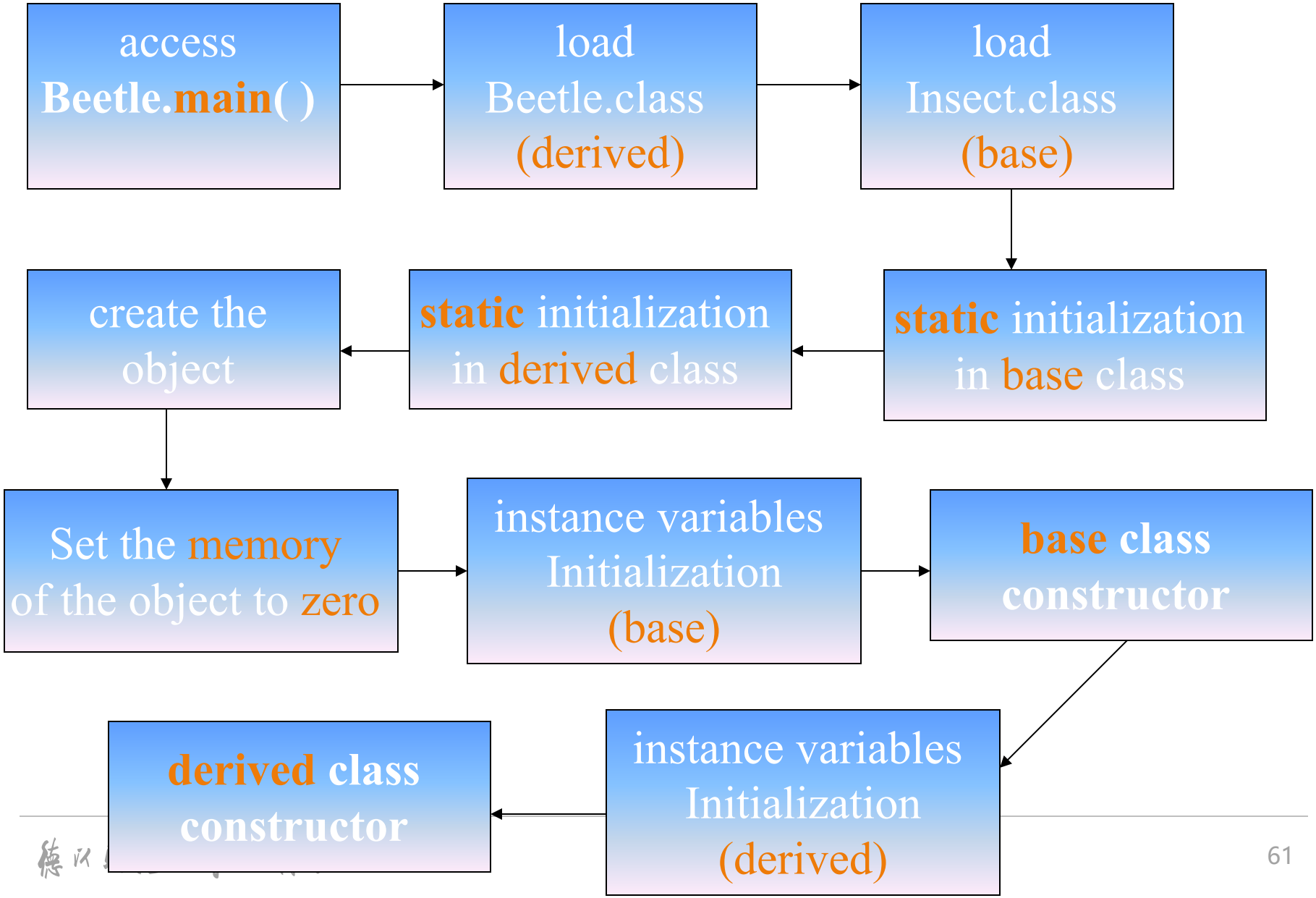
.

4.9初始化1

初始化包括类初始化和对象初始化

类初始化，有称为静态初始化，包括静态数据成员默认初始化和静态块初始化。仅发生在类加载时。

对象初始化，包括数据成员默认初始化、初始化块和构造器初始化。发生在类加载时。


access
Beetle.main( )
load
Beetle.class
(derived)
load
Insect.class
(base)
static initialization
in base class
static initialization
in derived class
create the
object
Set the memory
of the object to zero
instance variables
Initialization
(base)
base class
constructor
instance variables
Initialization
(derived)
derived class
constructor

4.9初始化1

类初始化发生在对象初始化之前

先为静态数据成员分配存储空间，执行默认初始化，然后执行静态块完成类的初始化。

静态数据成员的指定初始化是一种假象，因为编译时会成为静态块中的一条赋值语句，具体位置由它们在源码中的顺序决定。

Example：StaticInit.java

4.9初始化1

对象初始化先为数据成员分配存储空间，执行默认初始化，然后初始化块和构造器初始化。

在类编译后，对象初始化块会消失，其代码会被会被还原到每个构造器的所有代码之前。

可以将多个构造器内重用的代码放在对象代码块中，减少多个构造器内重用的代码

初始化块可以调用多次，随着对象的创建而调用

数据成员的指定初始化编译时会被还原到每个构造器的所有代码之前。

Example：ObjectInit.java

练习

1、写一个类，为该类添加文档注释、为该类的方法、成员变量添加文档注释。

在方法中添加单行注释并生成API文档

2、 创建一个名为Dog的类，带有重载的bark()方法。这个方法应该基于各种基本数据类型重载，并打印不同类型的吠叫，嚎叫等，这取决于调用哪个重载版本。编写一个main()函数，调用所有不同的版本。

3、创建一个具有两个方法的类。在第一个方法中，两次调用第二个方法：

第一次不使用this，第二次使用this。

4、创建一个具有两个(重载)构造函数的类。使用this，在第二个构造函数中调用第一个构造函数。